

Imports

```
In [256... from itertools import product

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.preprocessing import StandardScaler

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
import optuna
from optuna.samplers import TPESampler

import warnings
warnings.filterwarnings("ignore")

plt.style.use("ggplot")
sns.set_palette("husl")
```

Data Loading

```
In [257... items = pd.read_csv("items.csv")
item_categories = pd.read_csv("item_categories.csv")
sales_train = pd.read_csv("sales_train.csv")
shops = pd.read_csv("shops.csv")
test = pd.read_csv("test.csv")

In [258... # Preview all datasets
print(f"Sales Train: {sales_train.shape} – Daily transactions")
print(f"Items: {items.shape} – Item metadata")
print(f"Item Categories: {item_categories.shape} – Category names")
print(f"Shops: {shops.shape} – Shop names")
print(f"Test: {test.shape} – Month 34 predictions needed")
```

Sales Train: (2935849, 6) – Daily transactions
Items: (22170, 3) – Item metadata
Item Categories: (84, 2) – Category names
Shops: (60, 2) – Shop names
Test: (214200, 3) – Month 34 predictions needed

Preprocessing

```
In [259... # Check for duplicate IDs in all files
print("=== Duplicate Check ===")
print(f"Shops: {len(shops)} rows, {shops['shop_id'].nunique()} unique")
print(f"Items: {len(items)} rows, {items['item_id'].nunique()} unique")
```

```
print(f"Categories: {len(item_categories)} rows, {item_categories['item_c']
print(f"Sales: {len(sales_train):,} rows, {sales_train['shop_id'].nunique
```

=== Duplicate Check ===

Shops: 60 rows, 60 unique

Items: 22170 rows, 22170 unique

Categories: 84 rows, 84 unique

Sales: 2,935,849 rows, 60 shops, 21807 items

Fix duplicate shops (same physical location, different IDs): 57→0, 58→1, 11→10, 40→39

```
In [260... # Merge duplicate shops (map higher ID to lower ID)
shop_id_mapping = {57: 0, 58: 1, 11: 10, 40: 39}
sales_train["shop_id"] = sales_train["shop_id"].replace(shop_id_mapping)
print(f"Unique shops after merge: {sales_train['shop_id'].nunique()}")
```

Unique shops after merge: 56

Remove outliers from item_price and item_cnt_day before aggregation

```
In [261... # Remove outliers using IQR method
print(f"Before outlier removal: {len(sales_train):,} rows")

# Remove negative prices and extremely high prices (> 99.9th percentile)
price_upper = sales_train["item_price"].quantile(0.99)
price_mask = (sales_train["item_price"] > 0) & (sales_train["item_price"]

# Remove negative counts and extremely high daily counts (> 99.9th percen
cnt_upper = sales_train["item_cnt_day"].quantile(0.99)
cnt_mask = (sales_train["item_cnt_day"] >= 0) & (sales_train["item_cnt_da

sales_train = sales_train[price_mask & cnt_mask]
print(f"After outlier removal: {len(sales_train):,} rows")
print(f"Removed: {(~price_mask).sum() + (~cnt_mask).sum():,} outliers")
print(f"Price range: [{sales_train['item_price'].min():.2f}, {sales_train
print(f"Count range: [{sales_train['item_cnt_day'].min():,}, {sales_train['
```

Before outlier removal: 2,935,849 rows

After outlier removal: 2,872,935 rows

Removed: 63,482 outliers

Price range: [0.07, 5999.00]

Count range: [1.0, 5.0]

Aggregate daily → monthly and create full matrix with all (shop, item, month) combinations

```
In [262... # Step 1: Aggregate daily sales to monthly
monthly_sales = (
    sales_train
    .groupby(["date_block_num", "shop_id", "item_id"])
    .agg(item_cnt_month=("item_cnt_day", "sum"), item_price_avg=("item_pr
    .reset_index()
)
print(f"Monthly sales: {monthly_sales.shape[0]:,} rows")
```

Monthly sales: 1,592,043 rows

```
In [263... # Step 2: Create full matrix (all shop-item combinations per month)
matrix_parts = [
```

```
pd.DataFrame(
    list(product([m], d["shop_id"].unique(), d["item_id"].unique())),
    columns=["date_block_num", "shop_id", "item_id"]
)
for m, d in monthly_sales.groupby("date_block_num")
]
matrix = pd.concat(matrix_parts, ignore_index=True)
print(f"Full matrix: {matrix.shape[0]:,} rows")
```

Full matrix: 10,721,984 rows

```
In [264... # Step 3: Merge actual sales and add category info
matrix = (
    matrix
    .merge(monthly_sales, on=["date_block_num", "shop_id", "item_id"], ho
    .merge(items[["item_id", "item_category_id"]], on="item_id", how="lef
)
matrix["item_cnt_month"] = matrix["item_cnt_month"].fillna(0)
print(f"Matrix with sales: {matrix.shape}")
```

Matrix with sales: (10721984, 6)

Clip target to [0, 20] (competition requirement)

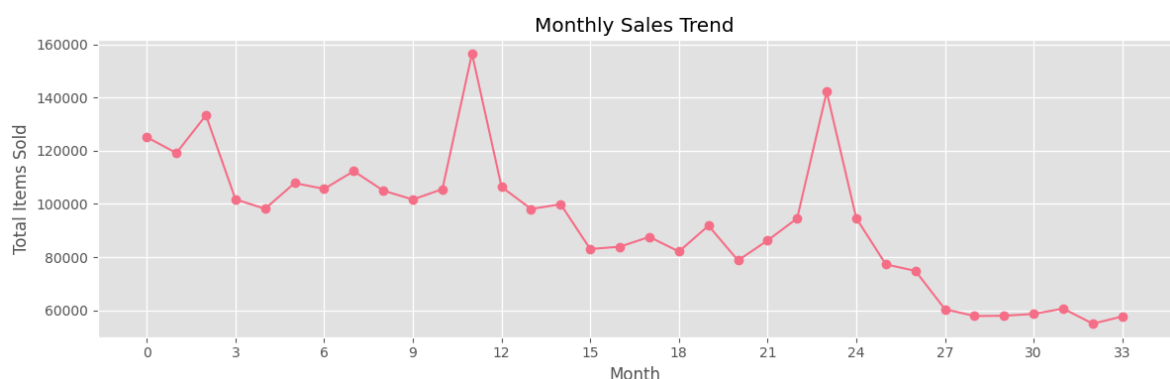
```
In [265... print(f"Before clip: [{matrix['item_cnt_month'].min()}, {matrix['item_cnt
matrix["item_cnt_month"] = matrix["item_cnt_month"].clip(0, 20)
print(f"After clip: [{matrix['item_cnt_month'].min()}, {matrix['item_cnt_
```

Before clip: [0.0, 106.0]

After clip: [0.0, 20.0]

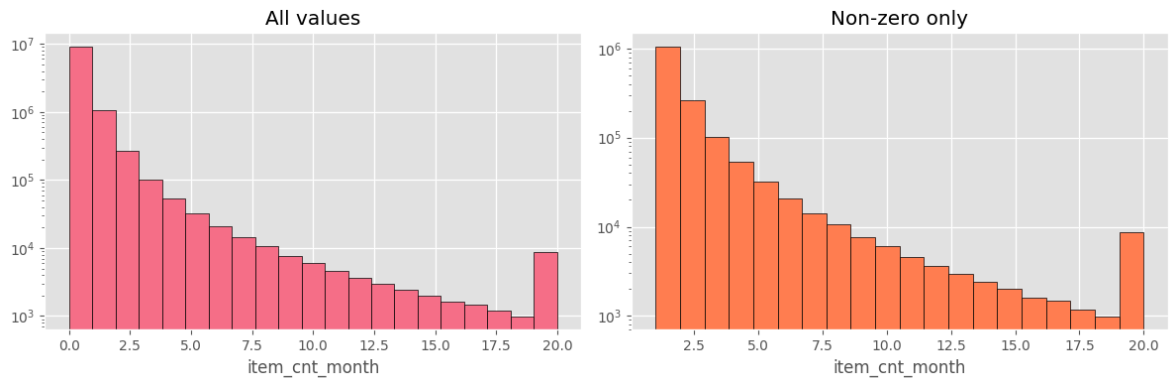
Exploratory Data Analysis

```
In [266... # Monthly sales trend
monthly_totals = matrix.groupby("date_block_num")["item_cnt_month"].sum()
plt.figure(figsize=(12, 4))
plt.plot(monthly_totals.index, monthly_totals.values, marker="o")
plt.xlabel("Month"); plt.ylabel("Total Items Sold"); plt.title("Monthly S
plt.xticks(range(0, 34, 3)); plt.tight_layout(); plt.show()
```



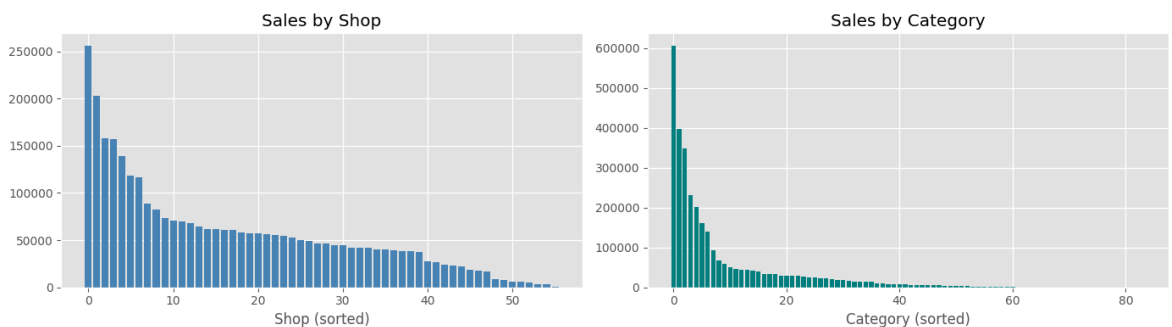
```
In [267... # Target distribution (85% are zeros)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(matrix["item_cnt_month"], bins=21, edgecolor="black"); axes[
axes[0].set_title("All values"); axes[0].set_xlabel("item_cnt_month")
non_zero = matrix[matrix["item_cnt_month"] > 0]["item_cnt_month"]
axes[1].hist(non_zero, bins=20, edgecolor="black", color="coral"); axes[1
axes[1].set_title("Non-zero only"); axes[1].set_xlabel("item_cnt_month")
```

```
plt.tight_layout(); plt.show()
print(f"Zero sales: {(matrix['item_cnt_month'] == 0).mean()*100:.1f}%")
```

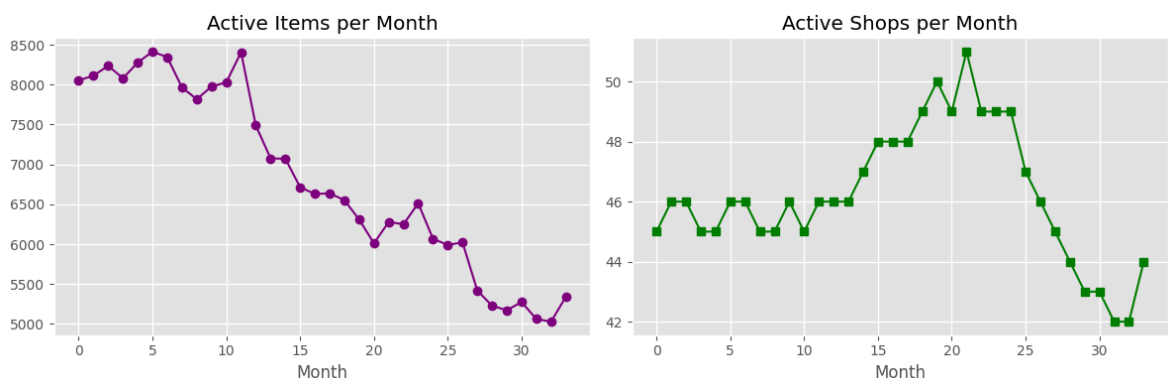


Zero sales: 85.2%

```
In [268... # Sales by shop and category
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
shop_sales = matrix.groupby("shop_id")["item_cnt_month"].sum().sort_value
axes[0].bar(range(len(shop_sales)), shop_sales.values, color="steelblue")
axes[0].set_title("Sales by Shop"); axes[0].set_xlabel("Shop (sorted)")
cat_sales = matrix.groupby("item_category_id")["item_cnt_month"].sum().so
axes[1].bar(range(len(cat_sales)), cat_sales.values, color="teal")
axes[1].set_title("Sales by Category"); axes[1].set_xlabel("Category (sor
plt.tight_layout(); plt.show()
```



```
In [269... # Active items/shops per month
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
active = matrix[matrix["item_cnt_month"] > 0]
axes[0].plot(active.groupby("date_block_num")["item_id"].nunique(), marke
axes[0].set_title("Active Items per Month"); axes[0].set_xlabel("Month")
axes[1].plot(active.groupby("date_block_num")["shop_id"].nunique(), marke
axes[1].set_title("Active Shops per Month"); axes[1].set_xlabel("Month")
plt.tight_layout(); plt.show()
```



```
In [270... # Dataset summary
print(f"Matrix: {matrix.shape[0]:,} rows × {matrix.shape[1]} cols")
```

```
print(f"Months: 0-33 (Jan 2013 - Oct 2015)")
print(f"Shops: {matrix['shop_id'].nunique()}, Items: {matrix['item_id'].nunique()}")
print(f"Target mean: {matrix['item_cnt_month'].mean():.3f}")
```

Matrix: 10,721,984 rows × 6 cols
 Months: 0-33 (Jan 2013 - Oct 2015)
 Shops: 56, Items: 21560, Categories: 84
 Target mean: 0.295

Handle Missing Values

```
In [271]: # Check and visualize missing values
missing_pct = (matrix.isnull().sum() / len(matrix) * 100).round(1)
print("Missing values (%):")
print(missing_pct[missing_pct > 0] if (missing_pct > 0).any() else "No missing values")
```

Missing values (%):
 item_price_avg 85.2
 dtype: float64

Fill missing `item_price_avg` with item's average price (then category average, then global average)

```
In [272]: # Fill missing prices: item avg → category avg → global avg
matrix["item_price_avg"] = matrix["item_price_avg"].fillna(
    matrix.groupby("item_id")["item_price_avg"].transform("mean")
)
matrix["item_price_avg"] = matrix["item_price_avg"].fillna(
    matrix.groupby("item_category_id")["item_price_avg"].transform("mean")
)
matrix["item_price_avg"] = matrix["item_price_avg"].fillna(matrix["item_price_avg"].mean())
print(f"Missing item_price_avg: {matrix['item_price_avg'].isnull().sum()}")
```

Missing item_price_avg: 0

Feature Engineering

```
In [273]: # Sort for lag creation and add month-of-year feature
matrix = matrix.sort_values(["shop_id", "item_id", "date_block_num"]).reset_index()
matrix["month"] = matrix["date_block_num"] % 12
print(f"Added month feature (0-11 for seasonality)")
```

Added month feature (0-11 for seasonality)

Lag features: sales from previous months (1, 2, 3, 6, 12)

```
In [274]: lag_features = []
for lag in [1, 2, 3, 6, 12]:
    col = f"item_cnt_month_lag_{lag}"
    matrix[col] = matrix.groupby(["shop_id", "item_id"])["item_cnt_month"].shift(lag)
    lag_features.append(col)
print(f"Created {len(lag_features)} lag features")
```

Created 5 lag features

Trend/momentum features: lag differences to capture sales acceleration/deceleration

```
In [275... # Trend features: differences between consecutive lags
trend_features = []
matrix["trend_lag1_lag2"] = matrix["item_cnt_month_lag_1"] - matrix["item_cnt_month_lag_0"]
matrix["trend_lag2_lag3"] = matrix["item_cnt_month_lag_2"] - matrix["item_cnt_month_lag_1"]
trend_features = ["trend_lag1_lag2", "trend_lag2_lag3"]
print(f"Created {len(trend_features)} trend features")
```

Created 2 trend features

Rolling means: average of last 3 and 6 months

```
In [276... rolling_features = []
for window in [3, 6]:
    col = f"item_cnt_month_roll_mean_{window}"
    matrix[col] = matrix.groupby(["shop_id", "item_id"])["item_cnt_month"].transform(
        lambda x: x.shift(1).rolling(window, min_periods=1).mean()
    )
    rolling_features.append(col)
print(f"Created {len(rolling_features)} rolling features")
```

Created 2 rolling features

Mean encoding: expanding average by shop, item, category

```
In [277... # Mean encoding (expanding average to avoid data leakage)
mean_features = []
for group_cols in ["shop_id", "item_id", "item_category_id", "shop_id", "item_id", "item_category_id"]:
    col = f"item_cnt_month_{group_cols}_avg"
    matrix[col] = matrix.groupby(group_cols)["item_cnt_month"].transform(
        lambda x: x.shift(1).expanding().mean()
    )
    mean_features.append(col)
print(f"Created {len(mean_features)} mean encoding features")
```

Created 4 mean encoding features

Fill NaN in lag/rolling/mean features with 0

```
In [278... new_features = lag_features + trend_features + rolling_features + mean_features
matrix[new_features] = matrix[new_features].fillna(0)
print(f"Filled {len(new_features)} features with 0")
```

Filled 13 features with 0

Recency features: first/last month item appeared, months since last sale

```
In [279... # Item first/last appearance in training (helps identify new vs established)
item_first_month = matrix.groupby("item_id")["date_block_num"].transform(
    lambda x: x.rolling(1).min()
)
item_last_month = matrix.groupby("item_id")["date_block_num"].transform(
    lambda x: x.rolling(1).max()
)

matrix["item_first_month"] = item_first_month
matrix["item_age"] = matrix["date_block_num"] - item_first_month # How long item has been in training
matrix["months_since_item_last_sale"] = matrix["date_block_num"] - item_last_month

# Shop-item pair first appearance
shop_item_first = matrix.groupby(["shop_id", "item_id"])["date_block_num"].transform(
    lambda x: x.rolling(1).min()
)
matrix["shop_item_first_month"] = shop_item_first
matrix["shop_item_age"] = matrix["date_block_num"] - shop_item_first
```

```
recency_features = ["item_age", "shop_item_age"]
print(f"Created {len(recency_features)} recency features")
```

Created 2 recency features

```
In [280... print(f"Final matrix: {matrix.shape[0]:,} rows x {matrix.shape[1]} column
print(f"Features: {matrix.columns.tolist()}")
```

Final matrix: 10,721,984 rows x 25 columns
Features: ['date_block_num', 'shop_id', 'item_id', 'item_cnt_month', 'item_price_avg', 'item_category_id', 'month', 'item_cnt_month_lag_1', 'item_cnt_month_lag_2', 'item_cnt_month_lag_3', 'item_cnt_month_lag_6', 'item_cnt_month_lag_12', 'trend_lag1_lag2', 'trend_lag2_lag3', 'item_cnt_month_roll_mean_3', 'item_cnt_month_roll_mean_6', 'item_cnt_month_shop_id_avg', 'item_cnt_month_item_id_avg', 'item_cnt_month_item_category_id_avg', 'item_cnt_month_shop_id_item_category_id_avg', 'item_first_month', 'item_age', 'months_since_item_last_sale', 'shop_item_first_month', 'shop_item_age']

Modeling

```
In [281... # Prepare training data - use only months >= 12 (recent data is more pred
MIN_TRAIN_MONTH = 12
matrix_train = matrix[matrix["date_block_num"] >= MIN_TRAIN_MONTH].copy()
print(f"Using months {MIN_TRAIN_MONTH}-33 for training (dropped early mon

feature_cols = [
    "shop_id", "item_id", "item_category_id", "month", "item_price_avg",
    *lag_features, *trend_features, *rolling_features, *mean_features, *r
]
X_train, y_train = matrix_train[feature_cols], matrix_train["item_cnt_mon
print(f"Training: {X_train.shape[0]:,} rows, {len(feature_cols)} features
```

Using months 12-33 for training (dropped early months with distribution shift)

Training: 6,275,726 rows, 20 features

```
In [282... # Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Prepare test set now (needed for evaluate_model)
test_df = test.copy()

# CRITICAL FIX: Apply same shop_id mapping as training data!
test_df["shop_id"] = test_df["shop_id"].replace(shop_id_mapping)

test_df["date_block_num"] = 34
test_df["month"] = 34 % 12
test_df = test_df.merge(items[["item_id", "item_category_id"]], on="item_id",
test_df["item_category_id"] = test_df["item_category_id"].fillna(test_df["

# Add features from training data
price_data = matrix[matrix["date_block_num"] == 33][["shop_id", "item_id"]
test_df = test_df.merge(price_data, on=["shop_id", "item_id"], how="left"

for lag in [1, 2, 3, 6, 12]:
    lag_data = matrix[matrix["date_block_num"] == 34 - lag][["shop_id", "
    lag_data.columns = ["shop_id", "item_id", f"item_cnt_month_lag_{lag}"]
    test_df = test_df.merge(lag_data, on=["shop_id", "item_id"], how="lef
```

```

# Trend features for test (computed from lags after fillna)
test_df["trend_lag1_lag2"] = test_df["item_cnt_month_lag_1"].fillna(0) -
test_df["trend_lag2_lag3"] = test_df["item_cnt_month_lag_2"].fillna(0) -

for window in [3, 6]:
    roll = matrix[matrix["date_block_num"].isin([33 - i for i in range(window)])]
    roll.columns = ["shop_id", "item_id", f"item_cnt_month_roll_mean_{window}"]
    test_df = test_df.merge(roll, on=["shop_id", "item_id"], how="left")

for cols in [
    ["shop_id", "item_id", "item_category_id", "shop_id", "item_id"],
    ["shop_id", "item_id", "item_category_id", "shop_id", "item_id"],
]:
    enc = matrix.groupby(cols)["item_cnt_month"].mean().reset_index()
    enc.columns = [*cols, f"item_cnt_month_{['_'].join(cols)}_avg"]
    test_df = test_df.merge(enc, on=cols, how="left")

# Recency features for test (month 34)
item_stats = matrix.groupby("item_id")["date_block_num"].agg(["min", "max"])
item_stats.columns = ["item_id", "item_first_month", "item_last_month"]
test_df = test_df.merge(item_stats, on="item_id", how="left")
test_df["item_age"] = 34 - test_df["item_first_month"].fillna(34)

shop_item_stats = matrix.groupby(["shop_id", "item_id"])["date_block_num"]
shop_item_stats.columns = ["shop_id", "item_id", "shop_item_first_month"]
test_df = test_df.merge(shop_item_stats, on=["shop_id", "item_id"], how="left")
test_df["shop_item_age"] = 34 - test_df["shop_item_first_month"].fillna(34)

# Drop intermediate columns
test_df = test_df.drop(columns=["item_first_month", "item_last_month", "shop_item_first_month"])

test_df = test_df.fillna(0)
X_test_scaled = scaler.transform(test_df[feature_cols])

print(f"Train: {X_train_scaled.shape}, Test: {X_test_scaled.shape}")

```

Train: (6275726, 20), Test: (214200, 20)

Evaluation Function (same as bikes.ipynb)

In [283]...

```

def evaluate_model(model, model_name, X_train, y_train, X_test):
    """Train model, evaluate on train, and predict on test. Same as bikes.ipynb"""
    model.fit(X_train, y_train)

    y_pred_train = np.clip(model.predict(X_train), 0, 20)
    y_pred_test = np.clip(model.predict(X_test), 0, 20)

    train_rmse = np.sqrt(mean_squared_error(y_train, y_pred_train))

    fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(18, 5))

    # Plot 1: Train actual vs predicted distribution
    # Sample for histogram (full data too large)
    sample_idx = np.random.choice(len(y_train), size=min(50000, len(y_train)))
    sns.histplot(y_train.iloc[sample_idx], kde=True, ax=axes[0], label="Actual")
    sns.histplot(y_pred_train[sample_idx], kde=True, ax=axes[0], label="Predicted")
    axes[0].set_title(f"{model_name}: Train Distribution")
    axes[0].set_xlabel("item_cnt_month")
    axes[0].legend()

    # Plot 2: Train actual vs predicted scatter (sample for visibility)

```



```

scatter_idx = np.random.choice(len(y_train), size=min(10000, len(y_train)), replace=True)
axes[1].scatter(y_train.iloc[scatter_idx], y_pred_train[scatter_idx],
               axes[1].plot([0, 20], [0, 20], "r--", lw=2)
axes[1].set_xlabel("Actual")
axes[1].set_ylabel("Predicted")
axes[1].set_title(f"{model_name}: Train Actual vs Predicted")

# Plot 3: Test predictions distribution
sns.histplot(y_pred_test, kde=True, ax=axes[2], color="green")
axes[2].set_title(f"{model_name}: Test Predictions Distribution")
axes[2].set_xlabel("Predicted item_cnt_month")

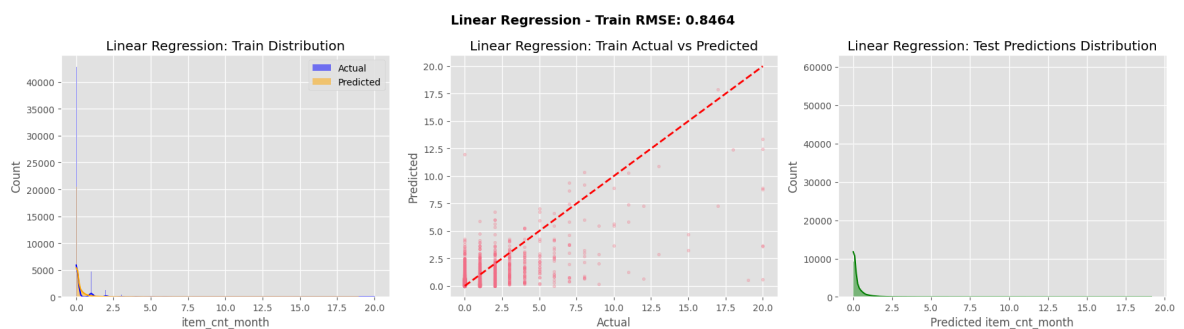
plt.suptitle(f"{model_name} - Train RMSE: {train_rmse:.4f}", fontsize=14)
plt.tight_layout()
plt.show()

return y_pred_train, y_pred_test, train_rmse

```

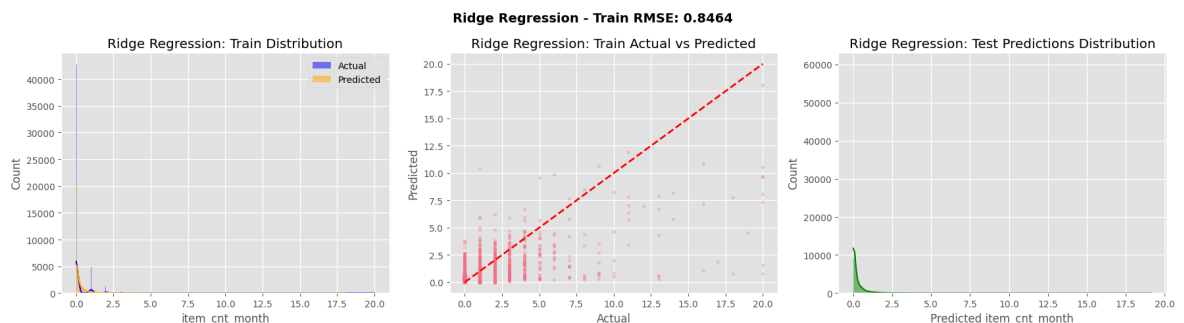
Linear Regression

In [284... `lr_model = LinearRegression()`
`y_pred_train_lr, y_pred_test_lr, rmse_lr = evaluate_model(lr_model, "Linear Regression")`



Ridge Regression

In [285... `# RidgeCV uses efficient leave-one-out CV internally`
`ridge_model = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100])`
`y_pred_train_ridge, y_pred_test_ridge, rmse_ridge = evaluate_model(ridge_model)`
`print(f"Best alpha: {ridge_model.alpha_}")`

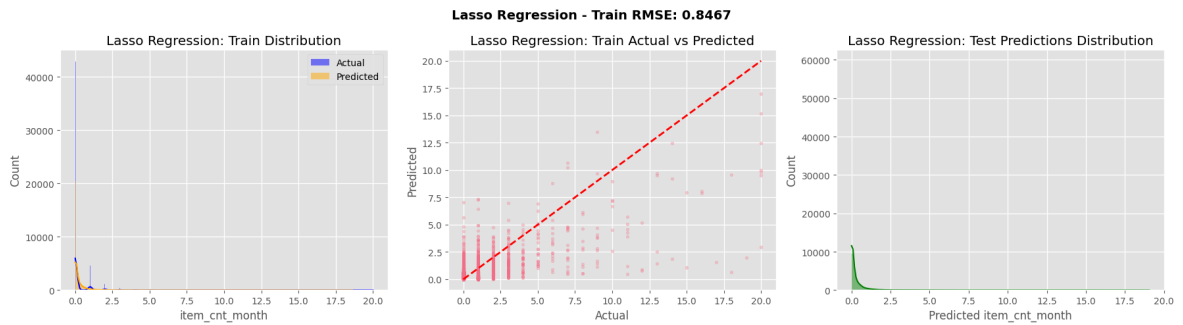


Best alpha: 100.0

Lasso Regression

In [286... `# LassoCV computes regularization path efficiently`
`lasso_model = LassoCV(alphas=[0.001, 0.01, 0.1, 1], cv=3, max_iter=1000)`
`y_pred_train_lasso, y_pred_test_lasso, rmse_lasso = evaluate_model(lasso_model)`

```
print(f"Best alpha: {lasso_model.alpha_:.4f}")
print(f"Features zeroed: {(lasso_model.coef_ == 0).sum()}/{len(feature_co
```



Best alpha: 0.0010
Features zeroed: 0/20

Random Forest (with Optuna)

```
In [287... # Use UNSCALED data for tree models + temporal validation split
# Validation = month 33 (closest to test month 34)
X_train_raw = X_train.values # Unscaled!
X_test_raw = test_df[feature_cols].values

# Use matrix_train (months >= 12) for validation split
val_mask = matrix_train["date_block_num"] == 33
train_mask = (matrix_train["date_block_num"] >= MIN_TRAIN_MONTH) & (matri

# Sample from training months only
sample_size = 500000
train_indices = np.where(train_mask)[0]
np.random.seed(42)
sample_idx = np.random.choice(train_indices, size=min(sample_size, len(tr

X_tr = X_train_raw[sample_idx]
y_tr = y_train.iloc[sample_idx].values

# Validation = all of month 33
X_val = X_train_raw[val_mask.values]
y_val = y_train[val_mask].values

print(f"Optuna: train sample {len(X_tr):,}, val (month 33) {len(X_val):,}

def rf_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 50, 150),
        "max_depth": trial.suggest_int("max_depth", 3, 10), # Lower = le
        "min_samples_split": trial.suggest_int("min_samples_split", 5, 20
        "min_samples_leaf": trial.suggest_int("min_samples_leaf", 5, 20),
        "random_state": 42,
        "n_jobs": -1
    }
    model = RandomForestRegressor(**params)
    model.fit(X_tr, y_tr)
    pred = np.clip(model.predict(X_val), 0, 20)
    return np.sqrt(mean_squared_error(y_val, pred))

optuna.logging.set_verbosity(optuna.logging.WARNING)
rf_study = optuna.create_study(direction="minimize", sampler=TPESampler(s
rf_study.optimize(rf_objective, n_trials=30, show_progress_bar=True)
```

```
print(f"Best Random Forest params: {rf_study.best_params}")
print(f"Best Val RMSE: {rf_study.best_value:.4f}")
```

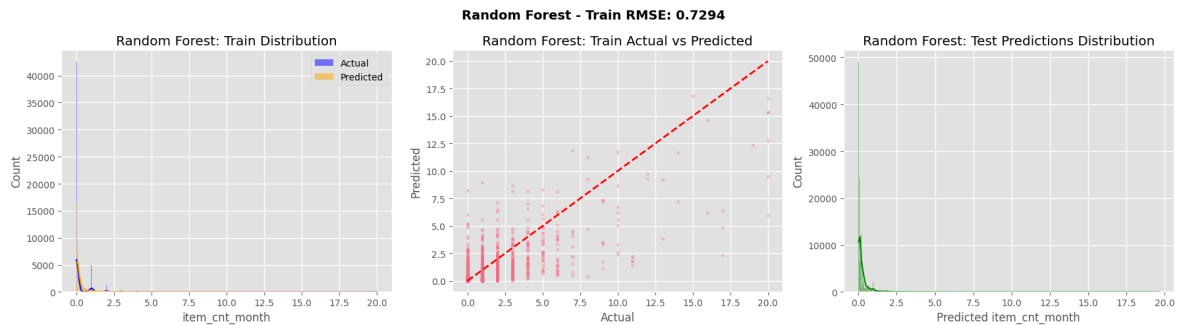
Optuna: train sample 500,000, val (month 33) 235,004

Best trial: 8. Best value: 0.729031: 100%|██████████| 30/30 [13:05<00:00, 26.19s/it]

Best Random Forest params: {'n_estimators': 56, 'max_depth': 10, 'min_samples_split': 20, 'min_samples_leaf': 17}

Best Val RMSE: 0.7290

```
In [288... # Train Random Forest with best params on full UNSCALED data
rf_model = RandomForestRegressor(**rf_study.best_params, random_state=42,
y_pred_train_rf, y_pred_test_rf, rmse_rf = evaluate_model(rf_model, "Rand
```



XGBoost (with Optuna)

```
In [289... # XGBoost with more regularization to prevent overfitting
def xgb_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 50, 200),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2,
        "max_depth": trial.suggest_int("max_depth", 2, 6), # Shallow tree
        "min_child_weight": trial.suggest_int("min_child_weight", 10, 100
        "subsample": trial.suggest_float("subsample", 0.6, 0.9),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6,
        "reg_alpha": trial.suggest_float("reg_alpha", 0.1, 10, log=True),
        "reg_lambda": trial.suggest_float("reg_lambda", 0.1, 10, log=True)
        "random_state": 42,
        "n_jobs": -1
    }
    model = XGBRegressor(**params)
    model.fit(X_tr, y_tr)
    pred = np.clip(model.predict(X_val), 0, 20)
    return np.sqrt(mean_squared_error(y_val, pred))

xgb_study = optuna.create_study(direction="minimize", sampler=TPESampler(
xgb_study.optimize(xgb_objective, n_trials=50, show_progress_bar=True)

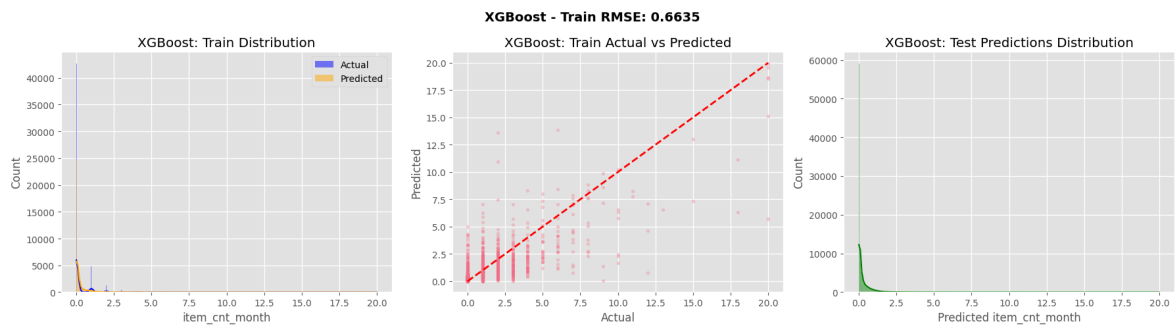
print(f"Best XGBoost params: {xgb_study.best_params}")
print(f"Best Val RMSE: {xgb_study.best_value:.4f}")
```

Best trial: 33. Best value: 0.687546: 100%|██████████| 50/50 [00:39<00:00, 1.27it/s]

Best XGBoost params: {'n_estimators': 190, 'learning_rate': 0.09738344039355167, 'max_depth': 6, 'min_child_weight': 26, 'subsample': 0.795539108492578, 'colsample_bytree': 0.8465201674764764, 'reg_alpha': 0.2631700206204546, 'reg_lambda': 4.461998701229907}

Best Val RMSE: 0.6875

```
In [290... # Train XGBoost with best params on full UNSCALED data
xgb_model = XGBRegressor(**xgb_study.best_params, random_state=42, n_jobs
y_pred_train_xgb, y_pred_test_xgb, rmse_xgb = evaluate_model(xgb_model, "
```



LightGBM (with Optuna)

```
In [291... # LightGBM with Optuna tuning
def lgbm_objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 50, 300),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2,
        "max_depth": trial.suggest_int("max_depth", 3, 8),
        "num_leaves": trial.suggest_int("num_leaves", 15, 127),
        "min_child_samples": trial.suggest_int("min_child_samples", 10, 1
        "subsample": trial.suggest_float("subsample", 0.6, 0.95),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6,
        "reg_alpha": trial.suggest_float("reg_alpha", 0.01, 10, log=True)
        "reg_lambda": trial.suggest_float("reg_lambda", 0.01, 10, log=Tru
        "random_state": 42,
        "n_jobs": -1,
        "verbosity": -1
    }
    model = LGBMRegressor(**params)
    model.fit(X_tr, y_tr)
    pred = np.clip(model.predict(X_val), 0, 20)
    return np.sqrt(mean_squared_error(y_val, pred))

lgbm_study = optuna.create_study(direction="minimize", sampler=TPESampler
lgbm_study.optimize(lgbm_objective, n_trials=50, show_progress_bar=True)

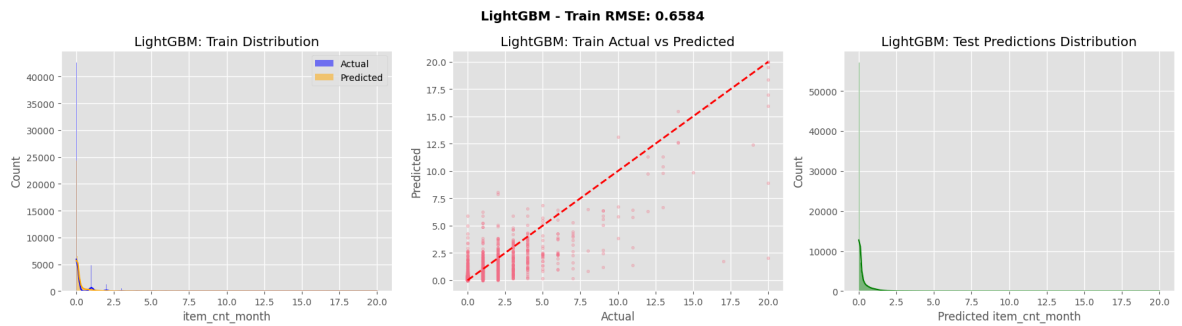
print(f"Best LightGBM params: {lgbm_study.best_params}")
print(f"Best Val RMSE: {lgbm_study.best_value:.4f}")
```

Best trial: 31. Best value: 0.679747: 100%|██████████| 50/50 [01:58<00:00, 2.36s/it]

Best LightGBM params: {'n_estimators': 299, 'learning_rate': 0.06457220748468775, 'max_depth': 8, 'num_leaves': 38, 'min_child_samples': 95, 'subsample': 0.6733611191999431, 'colsample_bytree': 0.7383193974591048, 'reg_alpha': 0.04649735005614882, 'reg_lambda': 0.010235329503182752}

Best Val RMSE: 0.6797

```
In [292... # Train LightGBM with best params on full UNSCALED data
lgbm_model = LGBMRegressor(**lgbm_study.best_params, random_state=42, n_j
y_pred_train_lgbm, y_pred_test_lgbm, rmse_lgbm = evaluate_model(lgbm_mode
```



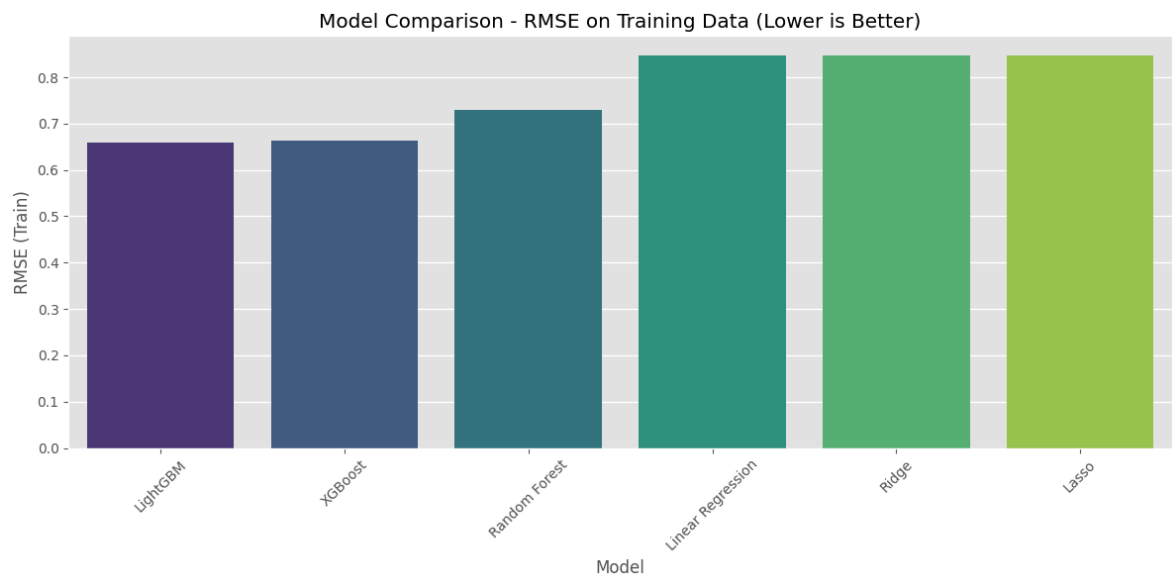
Model Comparison

```
In [293... # Model comparison (all 6 models)
results = {
    "Linear Regression": rmse_lr,
    "Ridge": rmse_ridge,
    "Lasso": rmse_lasso,
    "Random Forest": rmse_rf,
    "XGBoost": rmse_xgb,
    "LightGBM": rmse_lgbm
}

results_df = pd.DataFrame(list(results.items()), columns=["Model", "RMSE

fig, ax = plt.subplots(figsize=(12, 6))
sns.barplot(data=results_df, x="Model", y="RMSE (Train)", ax=ax, hue="Mod
ax.set_title("Model Comparison - RMSE on Training Data (Lower is Better)")
ax.tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()

print(results_df.to_string(index=False))
print(f"\nBest Model: {results_df.iloc[0]['Model']}")
```



Model	RMSE (Train)
LightGBM	0.658417
XGBoost	0.663531
Random Forest	0.729382
Linear Regression	0.846406
Ridge	0.846406
Lasso	0.846668

Best Model: LightGBM

Ensemble (XGBoost + Random Forest + LightGBM)

Averaging predictions from the three best tree models often reduces variance and improves generalization.

```
In [294... # Simple average ensemble of XGBoost, Random Forest, and LightGBM
y_pred_test_ensemble = np.clip((y_pred_test_xgb + y_pred_test_rf + y_pred
y_pred_train_ensemble = np.clip((y_pred_train_xgb + y_pred_train_rf + y_p

rmse_ensemble = np.sqrt(mean_squared_error(y_train, y_pred_train_ensemble

print(f"Ensemble Train RMSE: {rmse_ensemble:.4f}")
print(f"XGBoost Train RMSE: {rmse_xgb:.4f}")
print(f"RF Train RMSE: {rmse_rf:.4f}")
print(f"LightGBM Train RMSE: {rmse_lgbm:.4f}")
print(f"\nEnsemble test predictions: mean={y_pred_test_ensemble.mean():.3
```

```
Ensemble Train RMSE: 0.6733
XGBoost Train RMSE: 0.6635
RF Train RMSE: 0.7294
LightGBM Train RMSE: 0.6584
```

```
Ensemble test predictions: mean=0.321, std=0.785
```

Create Submission

Creating submission file using the best performing model (based on training RMSE).

```
In [295... # Store all predictions
all_predictions = {
    "linear": y_pred_test_lr,
    "ridge": y_pred_test_ridge,
    "lasso": y_pred_test_lasso,
    "random_forest": y_pred_test_rf,
    "xgboost": y_pred_test_xgb,
    "lightgbm": y_pred_test_lgbm,
    "ensemble": y_pred_test_ensemble
}

# Create submission file for each model
for name, pred in all_predictions.items():
    submission = pd.DataFrame({
        "ID": test_df["ID"],
        "item_cnt_month": pred
    })
    submission.to_csv(f"submission_{name}.csv", index=False)
    print(f"submission_{name}.csv: mean={pred.mean():.3f}")
```

```
# Also save best model as submission.csv
best_model_name = results_df.iloc[0]["Model"]
best_key = best_model_name.lower().replace(" ", "_")
best_pred = all_predictions[best_key]

submission = pd.DataFrame({
    "ID": test_df["ID"],
    "item_cnt_month": best_pred
})
submission.to_csv("submission.csv", index=False)

print(f"\n✓ Best model ({best_model_name}) saved as submission.csv")
print(f"\nSubmission preview:")
print(submission.head(10))
```

```
submission_linear.csv: mean=0.317
submission_ridge.csv: mean=0.317
submission_lasso.csv: mean=0.320
submission_random_forest.csv: mean=0.314
submission_xgboost.csv: mean=0.326
submission_lightgbm.csv: mean=0.324
submission_ensemble.csv: mean=0.321
```

✓ Best model (LightGBM) saved as submission.csv

Submission preview:

	ID	item_cnt_month
0	0	0.424953
1	1	0.063602
2	2	1.200221
3	3	0.318613
4	4	0.076569
5	5	0.639318
6	6	1.486422
7	7	0.083914
8	8	0.494091
9	9	0.216781

```
In [296... print("✓ Done! Upload any submission_*.csv to Kaggle for evaluation.")
print("\nSubmission files created:")
print(" - submission.csv (best model)")
print(" - submission_ensemble.csv ← TRY THIS FIRST (often best for gene
print(" - submission_linear.csv")
print(" - submission_ridge.csv")
print(" - submission_lasso.csv")
print(" - submission_random_forest.csv")
print(" - submission_xgboost.csv")
print(" - submission_lightgbm.csv")
```

✓ Done! Upload any submission_*.csv to Kaggle for evaluation.

Submission files created:

- submission.csv (best model)
- submission_ensemble.csv ← TRY THIS FIRST (often best for generalization)
- submission_linear.csv
- submission_ridge.csv
- submission_lasso.csv
- submission_random_forest.csv
- submission_xgboost.csv
- submission_lightgbm.csv